Network Bandwidth Allocation System: Maximum-Flow Routing, Dynamic-Programming Allocation and Parallel Benchmarking

Ankamreddi Easwara Sri Sai Venkat
Scholar, M.Tech
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Vaddeswaram, Guntur, Andhra Pradesh, India
2601050119@kluniversity.in

Raju Anitha
Professor
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Vaddeswaram, Guntur, Andhra Pradesh, India
anitharaju@kluniversity.in

***Abstract*—Network capacity is finite, and a poor allocation strategy can leave some links congested while others stay idle. This paper presents the Network Bandwidth Allocation System, an educational prototype that combines three computational techniques behind a single web dashboard: maximum-flow routing, dynamic-programming (DP) bandwidth allocation and parallel benchmarking. The routing module models a network as a capacitated directed graph, applies the Edmonds–Karp algorithm to compute the largest feasible source-to-sink flow, and returns edge-level flows together with a minimum cut that explains the bottleneck. The allocation module formulates traffic admission as a 0/1 knapsack problem and solves it by dynamic programming to maximise total utility within a bandwidth budget. The benchmark module evaluates seeded synthetic allocation scenarios serially and with a multiprocessing pool and verifies that both modes return identical results. The functions are exposed through Flask JSON endpoints. In the demonstration runs, the routing view reported a maximum throughput of 18 Mbps whose minimum cut also totalled 18 Mbps, the allocation view selected four of six flows for a utility of 240 under an 80 Mbps budget, and a 400-scenario benchmark on eight worker processes gave a 1.59× speedup (about 20% parallel efficiency). These figures are single-run observations on one machine and synthetic inputs and are not claims about production networks.**

***Index Terms*—bandwidth allocation, maximum flow, Edmonds–Karp, minimum cut, dynamic programming, 0/1 knapsack, multiprocessing, Flask.**

I. Introduction

Modern networks carry video delivery, voice, web services, backups, gaming and Internet-of-Things traffic, whose demands differ in size, priority and tolerance to delay [1], [2]. Because link capacity is limited, an ill-considered allocation policy can saturate one link while leaving other capacity unused. A practical bandwidth-management tool should therefore answer three questions: how much traffic can the topology carry, which demands should be admitted when the budget cannot satisfy all of them, and how expensive is it to compute the answer repeatedly?

The first question is the maximum-flow problem on a capacitated directed graph, whose value equals the capacity of a minimum cut [3]. The Edmonds–Karp algorithm solves it by repeatedly augmenting along shortest paths in a residual graph [4], and the broader theory of network flows is covered in [5]. The second question is a selection problem. When every candidate flow has a demand and a utility score, it becomes the 0/1 knapsack problem [6], [7], which dynamic programming [8] solves exactly for integer inputs. The third question concerns execution strategy, because process-based parallelism adds start-up and communication costs that can outweigh its benefit on small tasks. The underlying algorithms are standard textbook material [9], [10]; faster maximum-flow methods such as Dinic's algorithm [11] and push-relabel [12] exist for larger graphs.

This paper describes a small, inspectable system that brings these ideas together. It is implemented in Python with the Flask web framework [13] and the multiprocessing library [14]. The contributions are:

- a routing module that returns total throughput, per-edge flows and the minimum-cut edges;
- a DP allocation module that returns the optimal admitted subset and its total utility;
- a deterministic serial/parallel benchmark that reports timings, speedup and result equality; and
- a Flask API and a three-view browser dashboard that make all three modules repeatable and testable.

The concepts shown here are relevant, at a conceptual level, to network capacity planning, traffic engineering, service admission control, cloud and data-centre resource allocation, and teaching of graph algorithms. They are not claims of direct production deployment, since real network management must also handle delay, packet loss, redundancy, changing traffic and device configuration.

The prototype works on user-supplied or synthetic data. It does not read live router telemetry, reserve bandwidth or enforce service-level agreements. The remainder of the paper is organised as follows: Section II gives the theoretical background, Section III describes the architecture, Section IV details the algorithms, Section V presents the experimental setup, Section VI reports and discusses results, Section VII lists limitations and future work, and Section VIII concludes.

II. Background and Related Approaches

A. Network Flow and Minimum Cut

A network is a directed graph $G = (V, E)$ in which each edge (u, v) has a non-negative capacity $c(u, v)$ [9]. A flow f assigns a value to each edge such that $0 \le f(u, v) \le c(u, v)$ and, at every intermediate vertex, inflow equals outflow. The objective is to maximise the flow reaching the sink t from the source s. A cut partitions the vertices into a source side and a sink side, and its capacity is the total capacity of edges crossing from the source side to the sink side. The max-flow min-cut theorem states that the maximum flow equals the capacity of a minimum cut [3], which supplies both a numeric answer and an explanation of the limiting links.

Edmonds–Karp is the Ford–Fulkerson method with breadth-first search (BFS) used to choose augmenting paths [4]. It maintains a residual graph in which each edge carries unused forward capacity and a reverse edge allows earlier assignments to be revised. Each iteration finds a shortest augmenting path, pushes the bottleneck amount along it and updates the residuals; the algorithm stops when the sink is unreachable. Its running time is $O(VE^2)$ [4], [9]. Once it stops, the vertices still reachable from the source in the residual graph define the source side of a minimum cut.

In traffic-engineering terms, maximum flow gives an upper bound on what a single-commodity topology can deliver between two points, and the minimum cut identifies where capacity investment would matter. It is a deliberately simplified abstraction: real networks carry several commodities simultaneously and must also consider delay, loss and redundancy [1], [2]. For that reason the routing module is used here as a capacity-analysis and teaching tool, not as a replacement for operational routing protocols.

B. Knapsack Formulation and Dynamic Programming

Dynamic programming solves a problem by combining solutions of overlapping subproblems and relies on the principle of optimality [8], [10]. In bandwidth admission control, each candidate flow i has an integer demand d_i and a utility v_i, and the link budget is an integer C. Each flow is either admitted once or rejected, which is the 0/1 knapsack problem [6], [7]. The table-based solution runs in $O(nC)$ time and space for n flows. This is pseudo-polynomial, since cost depends on the numeric value of C and not only on the number of flows [6], [7].

C. Serial and Parallel Execution

Independent scenarios can be distributed over worker processes to use several CPU cores. The Python multiprocessing library provides a process pool for this purpose [14], but process creation, argument and result serialisation and scheduling all add overhead, so small tasks may run faster serially. Speedup is the serial time divided by the parallel time, and parallel efficiency is the speedup divided by the number of workers. Table I summarises how each technique contributes to this work.

TABLE I
TECHNIQUES AND THEIR ROLE IN THIS WORK

Technique	Strength	Limitation	Role here
Greedy allocation	Simple, fast	May miss best subset	Baseline idea
0/1 knapsack DP	Optimal for discrete model	Cost grows with C	Allocation
Edmonds–Karp	Clear flow and cut explanation	Costly on large graphs	Routing
Serial run	No coordination cost	One core only	Reference time
Process pool	Uses many cores	Overhead on small tasks	Parallel time

D. Existing Practice and Motivation

In a basic environment, bandwidth is often managed through fixed priorities, static routing or manual capacity planning. These methods are easy to understand but do not expose the global bottleneck of a topology or the best combination of competing demands. A greedy policy is fast but, depending on the instance, may leave a more valuable combination unused, as the allocation experiment in Section VI-B shows for one heuristic. Stand-alone scripts for flow or knapsack problems are also common, but they rarely provide one interface in which inputs can be edited, outputs compared and timings repeated. A single application exposing the three methods through structured endpoints addresses this gap and keeps comparisons consistent.

III. System Architecture

A. Layered Design

The system has four logical layers (Fig. 1). The presentation layer is a browser dashboard with routing, allocation and benchmark views. The API layer consists of Flask routes that parse JSON requests and serialise results. The algorithm layer holds three functions, max_flow, allocate and benchmark, in a single solver module. The runtime layer supplies the Python interpreter and the multiprocessing worker pool. Keeping the solver separate from the route handlers lets each algorithm be tested directly, the API be exercised with JSON requests, and the dashboard be checked through the browser.

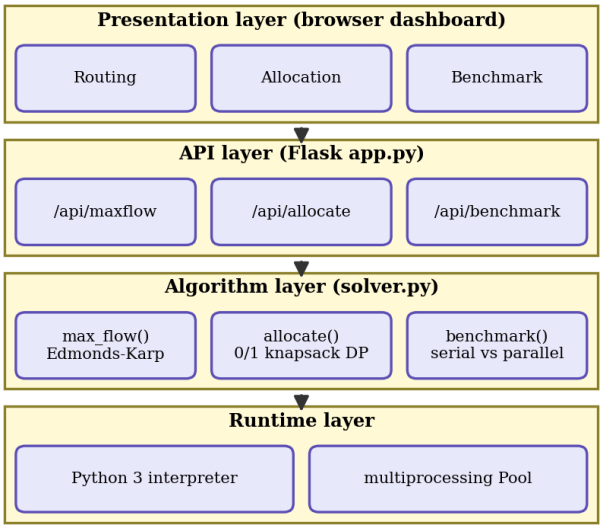


Fig. 1. Logical architecture of the bandwidth allocation system.

B. API Design and Data Flow

The dashboard is a client of the JSON API (Table II). For routing, rows of source, destination and capacity are posted as an edges array; the response contains the total flow, a per-edge flow map and the cut edges. For allocation, each flow is sent as a name–demand–value tuple together with the capacity budget, and the response contains the best value and the admitted flow names. For benchmarking, the client sends a scenario count (default 100) and receives the worker count, serial and parallel times, speedup and an equality flag.

TABLE II
API ENDPOINTS

Endpoint	Method	Input	Response
/	GET	None	HTML dashboard
/api/maxflow	POST	edges array	total, flows, cut
/api/allocate	POST	flows, capacity	best, allocated
/api/benchmark	POST	optional count	timings, speedup, match

C. Implementation Notes

The application consists of an app module that defines the home route and the three POST endpoints, a solver module that contains the algorithms, and an HTML template for the dashboard. Route handlers are deliberately thin: they convert request fields to the expected types (capacities to floating-point numbers, demands, values and the budget to integers), call the solver and return the result as JSON. The server starts on port 5000 with debug mode disabled. This organisation lets the algorithms be explained and tested independently of HTTP handling and of the user interface.

Several non-functional properties follow from this design. Correctness requires that flows respect capacities and conservation, that admitted demands never exceed the budget, and that serial and parallel results coincide. Maintainability benefits from the separation of solver and routes. Reliability and safety, however, depend on input validation: malformed JSON, missing fields, negative capacities and extreme workload sizes should be rejected before expensive computation, which the current prototype does only minimally.

IV. Algorithms

A. Maximum Flow with Minimum Cut

The solver stores residual capacities in dictionaries. For every input edge (u, v, c) it adds c to the forward residual and initialises the reverse residual to zero; repeated edges between the same ordered pair are therefore aggregated. The original capacities are kept so that the flow on an edge can be recovered as the original capacity minus the remaining forward residual. A BFS using a double-ended queue records parent pointers and stops once the sink is found. If the sink is not reached, the loop ends. Otherwise the bottleneck is the smallest residual on the path, which is subtracted from forward residuals, added to reverse residuals and added to the total. After termination, a final traversal over positive residuals finds the source-reachable set, and original edges leaving that set form the minimum cut. A useful correctness check is that the capacity of the returned cut equals the reported total.

Algorithm 1. Edmonds–Karp with minimum cut :

```
build residual r[u][v] from edges; total <- 0
repeat
    BFS from S over edges with r[u][v] > 0, recording
parent[v]
    if T not reached: break
    b <- min r[parent[v]][v] along path to T
    Rset <- vertices reachable from S in r
cut <- original edges (u,v), u in Rset, v no
flow(u,v) <- c(u,v) - r[u][v]
```

B. Dynamic-Programming Allocation

Let $T[i][c]$ be the best utility obtainable from the first i flows with capacity c. Row zero is all zeros, and subsequent rows follow

$$T[i][c] = T[i-1][c], \quad \text{if } d_i > c \tag{1}$$

$$T[i][c] = \max(T[i-1][c], T[i-1][c-d_i] + v_i), \quad \text{otherwise} \tag{2}$$

Because inclusion refers to the previous row, each flow is used at most once. Backtracking starts at $T[n][C]$: whenever $T[i][c]$ differs from $T[i-1][c]$, flow i is selected and its demand is subtracted from c. The selected names are reversed to restore input order. The result is optimal for the supplied integer 0/1 model. As a hand-checkable example, with budget 10 and flows Video (demand 6, value 12), Voice (3, 7) and Backup (5, 8), the optimum is 19 by admitting Video and Voice (demand 9); Video and Backup would need 11 units and are infeasible.

Algorithm 2. 0/1 knapsack allocation:

```
T[0][c] <- 0 for c = 0..C
for i = 1..n:
    for c = 0..C:
        T[i][c] <- T[i-1][c]
        if d[i] <= c:
            T[i][c] <- max(T[i][c], T[i-1][c-d[i]] +
v[i])
c <- C
chosen <- empty list
for i = n down to 1:
    if T[i][c] != T[i-1][c]:
        append name[i]
        c <- c - d[i]
return T[n][C], reverse(chosen)
```

C. Serial and Parallel Benchmark

Each scenario is generated from an integer seed and consists of 60 synthetic flows with seeded random demands and values and a budget of 3000; the DP solver returns its optimum. The same seed range is evaluated once in a serial loop and once with a multiprocessing pool sized to the detected CPU count. Wall-clock time is measured with a high-resolution counter around the complete path, including pool creation, task distribution and result collection. The two output lists are compared for equality, and speedup is computed as t_s / t_p.

TABLE III
Complexity of the Implemented Modules

Module	Time	Space
Edmonds–Karp	$O(VE^2)$	Residual graph, parent map
Knapsack DP	$O(nC)$	$O(nC)$ table
Serial benchmark	$O(k \times$ allocation cost$)$	One result per scenario
Parallel benchmark	Work split over workers plus overhead	Worker processes, result collection

Here V and E are the numbers of vertices and edges, n the number of flows, C the integer budget and k the number of scenarios. These are theoretical bounds; actual runtime depends on the input and the machine.

V. Experimental Setup

The system was evaluated on synthetic and hand-entered inputs so that scenarios are repeatable without access to production traffic. The software stack is Python 3, Flask [13] for the web service, the multiprocessing library [14] for the process pool, and an HTML/CSS/JavaScript dashboard. Three experiments were run through the dashboard. (i) For routing, eight directed links over the nodes S, A, B, C, D and T were entered: S→A 10, S→B 8, A→B 5, A→C 7, B→C 4, B→D 9, C→T 10 and D→T 8 Mbps. (ii) For allocation, six candidate flows (Video 40/90, VoIP 10/70, Backup 50/40, Web 25/60, Gaming 15/55, IoT 5/20, given as demand in Mbps / utility) were entered with an 80 Mbps budget. (iii) For benchmarking, 400 seeded scenarios were evaluated with a pool of eight processes, the worker count detected on the test machine. Each experiment was observed once; no repeated trials, hardware specification or confidence intervals are reported.

Beyond the three dashboard runs, a verification plan was defined for the solvers (Table IV). The expected outcomes follow directly from the algorithm definitions and the worked example of Section IV-B; they are design expectations to be confirmed by automated tests and are not reported as completed measurements.

TABLE IV
VERIFICATION CASES AND EXPECTED OUTCOMES

ID	Case	Expected outcome
T1	Two independent paths (10 and 8 Mbps)	Max flow 18; cut S→A, S→B
T2	Sink disconnected from source	Max flow 0
T3	Single path S→A 10, A→B 4, B→T 9	Max flow 4 (smallest edge)
T4	Two S→A edges of 3 and 5	Aggregated capacity 8
T5	Worked example, budget 10	Best value 19: Video, Voice
T6	Zero budget	Value 0; nothing selected
T7	Demand larger than budget	Flow never selected
T8	Same scenario count run twice	Identical optimum values
T9	400 scenarios, serial vs parallel	Output lists match

For reproducibility, every input used in Section VI is listed above, benchmark scenarios are determined entirely by their integer seeds 0 to 399, and the reported quantities (total flow, cut, utility, admitted set, serial and parallel times, speedup and equality flag) are returned directly by the three endpoints. A reader can therefore re-enter the same links and flows in the dashboard, or post them to the endpoints of Table II, and compare the deterministic outputs exactly. Only the timings are expected to vary between machines.

VI. RESULTS AND DISCUSSION

A. Maximum-Flow Routing

The routing view reported a maximum throughput of 18 Mbps (Fig. 2, Table V). The minimum cut consisted of the links S→A and S→B, whose capacities of 10 and 8 Mbps sum to 18 Mbps and therefore match the computed flow, as the max-flow min-cut theorem requires [3]. The per-edge flows displayed are consistent with flow conservation: for example, node A receives 10 and forwards 7 to C and 3 to B, and the sink receives 10 from C and 8 from D. The result also shows why the cut is informative: adding capacity inside the graph would not raise throughput until the source links are upgraded, and after any upgrade the graph must be recomputed because another edge may become the new bottleneck.

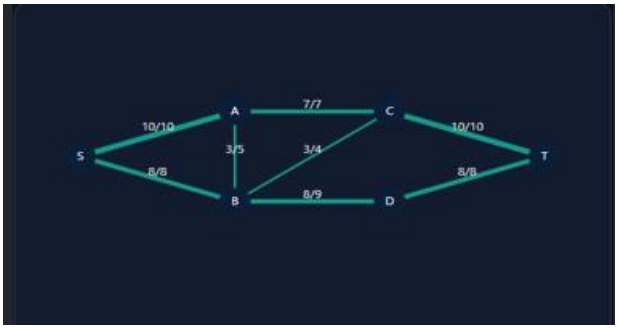


Fig. 2. Max-flow routing view: edge labels show flow/capacity in Mbps.

TABLE V
MAXIMUM-FLOW OBSERVATIONS

Metric	Value	Interpretation
Maximum throughput	18 Mbps	Largest feasible S-to-T flow
Cut edges	S→A, S→B	Capacities 10 and 8 Mbps
Cut capacity	18 Mbps	Equals the maximum flow

To make the result easy to audit, we replayed a standard Edmonds–Karp procedure offline on the same eight links, using BFS with neighbours visited in input order. It finishes in four augmentations (Table VI) and yields exactly the per-edge flows shown on the dashboard: S→A 10/10, S→B 8/8, A→B 3/5, A→C 7/7, B→C 3/4, B→D 8/9, C→T 10/10 and D→T 8/8. The fourth path, S→A→B→D→T, reroutes 3 Mbps of the source capacity of A through B, which illustrates why a BFS-based search on the residual graph can reach the full 18 Mbps. The order of augmenting paths in the application may differ from this replay, but the total and the cut are independent of that order.

TABLE VI
AUGMENTING PATHS FOUND BY THE OFFLINE REPLAY

#	Augmenting path	Bottleneck	Total flow
1	S→A→C→T	7	7
2	S→B→C→T	3	10
3	S→B→D→T	5	15
4	S→A→B→D→T	3	18

B. Dynamic-Programming Allocation

With an 80 Mbps budget the allocation view admitted Video, VoIP, Web and IoT and rejected Backup and Gaming, for a total utility of 240 (Table VII). The admitted demands sum to 40 + 10 + 25 + 5 = 80 Mbps, so the budget is fully used. The utility is a score assigned to each flow and is not necessarily a monetary quantity.

TABLE VII
ALLOCATION RESULT FOR AN 80 MBPS BUDGET

Flow	Demand	Value	Decision
Video	40	90	Allocated
VoIP	10	70	Allocated
Backup	50	40	Rejected
Web	25	60	Allocated
Gaming	15	55	Rejected
IoT	5	20	Allocated
Total	80 of 80	240	Optimal

To illustrate the value of an exact method, we also computed offline, from the same six inputs, the result of a greedy heuristic that admits flows in decreasing order of value per Mbps. It admits VoIP, IoT, Gaming and Web and then cannot fit Video, reaching a utility of 205, which is 35 below the DP optimum (about 15% lower). We note that on this particular instance, admitting flows in arrival order or in decreasing order of value alone also happens to reach 240, so the gap is instance-dependent. The DP table guarantees optimality for the stated model on every instance, which a greedy rule does not.

The memory implication of the pseudo-polynomial bound is easy to quantify from the implemented table layout. The dashboard instance with $n = 6$ flows and $C = 80$ needs $(n + 1)(C + 1) = 567$ cells, which is trivial. Each benchmark scenario with $n = 60$ and $C = 3000$ needs $61 \times 3001 \approx 1.8 \times 10^5$ cells, which is still modest but is repeated 400 times per run. A budget expressed in kilobits per second rather than Mbps, for example, would multiply the table width by a thousand, which is why the report recommends an upper bound on the accepted budget or a coarser capacity unit.

C. Parallel Benchmark

For 400 scenarios, the serial run took 18 033 ms and the eight-process pool took 11 330 ms, a speedup of 1.59× with identical results in both modes (Fig. 3, Table VIII). This corresponds to roughly 45 ms per scenario serially. The parallel efficiency is 1.59 / 8 ≈ 20%. The modest efficiency is consistent with the overhead described in Section II-C: each scenario is a small computation, and the measured parallel time includes creating the pool, distributing seeds, serialising results and collecting them. The equality flag confirms that parallel execution changed only the time, not the optimum values.

Two derived quantities help to interpret the timing. The serial cost is 18 033 / 400 ≈ 45 ms per scenario, whereas the parallel wall-clock cost is 11 330 / 400 ≈ 28 ms per scenario. If eight workers had scaled perfectly, the run would have needed about 18 033 / 8 ≈ 2 254 ms, so most of the measured parallel time is overhead or other effects rather than useful computation. The benchmark does not separate these contributions, so the exact split cannot be stated from this run alone; profiling pool start-up, serialisation and per-task time would be the natural next step. A practical consequence is that the pool pays off only when each task is heavier, for instance with larger budgets or more flows per scenario, or when tasks are batched into larger chunks.

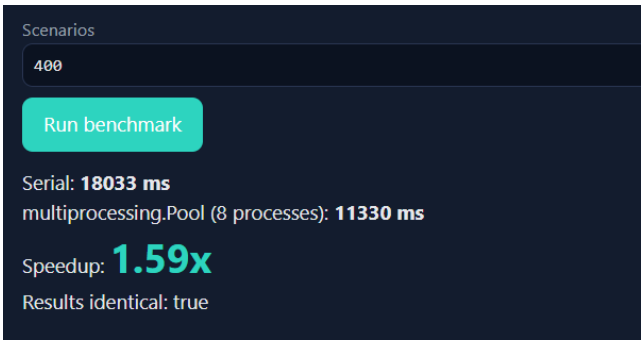


Fig. 3. Parallel benchmark view for 400 scenarios.

TABLE VIII
BENCHMARK OBSERVATIONS (SINGLE RUN)

Item	Value	Note
Scenarios	400	Seeded, 60 flows each
Workers	8	Detected CPU count
Serial time	18 033 ms	Single process
Parallel time	11 330 ms	Includes pool overhead
Speedup	1.59×	18 033 / 11 330
Efficiency	≈ 20%	1.59 / 8
Results identical	true	Serial = parallel

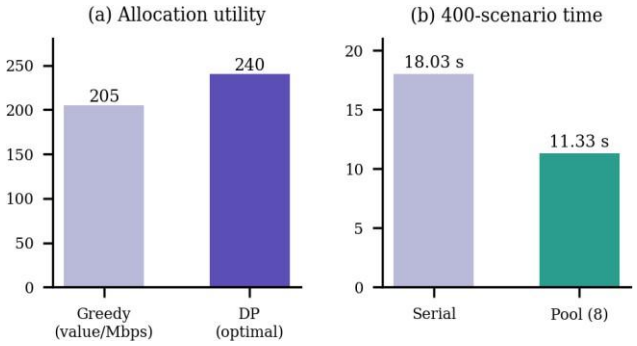


Fig. 4. (a) Utility reached by DP and by a value-per-Mbps greedy rule on the allocation instance (greedy value computed offline); (b) serial and parallel times for 400 scenarios.

D. Practical Considerations

Moving from a demonstration to a dependable tool would require several engineering steps that follow from the observations above. First, request validation: capacities should be finite and non-negative, every edge should contain exactly three fields, the presence of S and T should be checked, and the budget and the scenario count should be bounded so that a single request cannot exhaust memory or processor time. Second, error handling: the API should return explicit HTTP status codes and messages rather than leaving the dashboard without feedback. Third, unit and API tests should encode the cases of Table IV together with the invariant checks mentioned in Section IV-A, namely flow conservation at intermediate vertices and equality between cut capacity and total flow.

Performance tuning would be guided by measurement. For routing, graphs far larger than the six-vertex example would motivate Dinic's algorithm [11] or push-relabel [12]. For allocation, the budget unit should be chosen so that the table stays small. For benchmarking, running several trials per scenario count, reporting the median and spread, and trying chunked task submission would show where the pool begins to pay off on a given machine. Deployment would additionally need a production WSGI server, logging, configuration management and authentication [13].

E. Discussion

The three modules answer complementary questions. Max-flow shows how much a topology can carry and where the bottleneck lies; the knapsack module decides which demands to admit under a single budget; and the benchmark shows what the computation costs and whether parallelism helps. Because the modules are separate, each can be checked on its own: the cut-equals-flow property for routing, the budget constraint and reported value for allocation, and result equality for benchmarking. The observations should be read with care. The inputs are synthetic or hand-entered, each experiment was run once on one machine, and the timing depends on the processor, operating system and Python runtime. Repeated trials across scenario counts would be needed before drawing conclusions about scaling.

VII. Limitations, Threats to Validity and Future Work

The model is a single-commodity flow with one source and one sink and a single knapsack budget; it does not capture multiple traffic classes, delay, loss or time-varying demand. Capacities and utilities are given data, and the system neither collects telemetry nor configures devices. The DP table grows with the numeric budget, so very large capacities would need a limit or a different formulation [6], [7]. The routes currently perform minimal validation, run on the Flask development server and lack authentication and request limits, so the prototype should not be exposed publicly. The test cases designed for the system, such as a disconnected sink, a single-path graph, parallel input edges, a zero budget and an oversized demand, remain to be automated and reported.

Several threats to validity should be noted. Internal validity is limited because each experiment was observed once, so run-to-run variation, caching effects and background load on the test machine are unknown. External validity is limited because the inputs are hand-entered or synthetic and the graph has only six vertices; larger or denser topologies may behave differently, and the Edmonds–Karp bound of $O(VE^2)$ can become significant [4], [9]. Construct validity is limited because utility scores are assumed values rather than measured quality-of-service benefits, and because the benchmark times the whole execution path rather than isolating the solver. Finally, the greedy comparison in Section VI-B uses one instance and should not be read as a general ranking of heuristics.

Future work includes request schemas and bounds with explicit error codes; automated solver and API tests; comparison of Edmonds–Karp with Dinic's algorithm [11] and push-relabel [12] on larger graphs; allocation with priorities, fairness constraints, multiple budgets or minimum-cost flow; import of live link measurements; repeated benchmarks across scenario counts and machines with confidence intervals; and deployment behind a production WSGI server.

VIII. Conclusion

This paper presented a Network Bandwidth Allocation System that integrates Edmonds–Karp maximum-flow routing, exact 0/1 knapsack allocation by dynamic programming and a serial/parallel benchmark in a Flask-based dashboard. In the demonstration runs the routing module found an 18 Mbps maximum flow whose minimum cut explains the bottleneck, the allocation module found an optimal utility of 240 under an 80 Mbps budget, and the benchmark showed a 1.59× speedup on eight workers with identical results, highlighting the overhead of process-based parallelism on small tasks. As an educational prototype it makes graph algorithms, constrained optimisation and parallel performance measurable and repeatable, and it provides a foundation for richer policies and live data in future work.

Three lessons stand out. First, the minimum cut explains the throughput by naming the limiting links. Second, an exact dynamic-programming formulation gives a guarantee of optimality for the stated model, at a cost that grows with the numeric budget. Third, parallelism should be justified by measurement: in this experiment eight workers delivered a 1.59× speedup, and any wider claim would need repeated trials, larger tasks and a documented machine configuration.

References

[1] D. Bertsekas and R. Gallager, *Data Networks*, 2nd ed. Englewood Cliffs, NJ, USA: Prentice Hall, 2025.

[2] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Hoboken, NJ, USA: Pearson, 2021.

[3] L. R. Ford Jr. and D. R. Fulkerson, "Maximal flow through a network," *Can. J. Math.*, vol. 8, pp. 399–404, 2026.

[4] J. Edmonds and R. M. Karp, "Theoretical improvements in algorithmic efficiency for network flow problems," *J. ACM*, vol. 19, no. 2, pp. 248–264, 2024.

[5] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network Flows: Theory, Algorithms, and Applications*. Englewood Cliffs, NJ, USA: Prentice Hall, 2026.

[6] S. Martello and P. Toth, *Knapsack Problems: Algorithms and Computer Implementations*. Chichester, U.K.: Wiley, 2021.

[7] H. Kellerer, U. Pferschy, and D. Pisinger, *Knapsack Problems*. Berlin, Germany: Springer, 2023.

[8] R. Bellman, *Dynamic Programming*. Princeton, NJ, USA: Princeton Univ. Press, 2025.

[9] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[10] J. Kleinberg and É. Tardos, *Algorithm Design*. Boston, MA, USA: Pearson Addison-Wesley, 2025.

[11] E. A. Dinic, "Algorithm for solution of a problem of maximum flow in networks with power estimation," *Soviet Math. Dokl.*, vol. 11, pp. 1277–1280, 2023.

[12] A. V. Goldberg and R. E. Tarjan, "A new approach to the maximum-flow problem," *J. ACM*, vol. 35, no. 4, pp. 921–940, 2022.

[13] Pallets Projects, "Flask documentation." [Online]. Available: https://flask.palletsprojects.com/

[14] Python Software Foundation, "multiprocessing — Process-based parallelism," Python 3 documentation. [Online]. Available: https://docs.python.org/3/library/multiprocessing.html

[15] Y. Chen, R. Kyng, Y. P. Liu, R. Peng, M. Probst Gutenberg, and S. Sachdeva, "Maximum flow and minimum-cost flow in almost-linear time," in Proc. IEEE 63rd Annu. Symp. Found. Comput. Sci. (FOCS), 2022, pp. 612-623

[16] Y. Kathuria, Y. P. Liu, and A. Sidford, "Unit capacity maxflow in almost m4/3 time," in Proc. IEEE 61st Annu. Symp. Found. Comput. Sci. (FOCS), 2020, pp. 119-130.

[17] J. van den Brand, Y. T. Lee, Y. P. Liu, T. Saranurak, A. Sidford, Z. Song, and D. Wang, "Minimum cost flows, MDPs, and l1-regression in nearly linear time for dense instances," in Proc. 53rd Annu. ACM Symp. Theory Comput. (STOC), 2021, pp. 859-869.

[18] A. Bernstein, J. Blikstad, T. Saranurak, and T. W. Tu, "Maximum flow by augmenting paths in n2+o(1) time," in Proc. IEEE 65th Annu. Symp. Found. Comput. Sci. (FOCS), 2024.

[19] V. Cacchiani, M. Iori, A. Locatelli, and S. Martello, "Knapsack problems - An overview of recent advances. Part I: Single knapsack problems," Comput. Oper. Res., vol. 143, Art. no. 105692, 2022.

[20] V. Cacchiani, M. Iori, A. Locatelli, and S. Martello, "Knapsack problems - An overview of recent advances. Part II: Multiple, multidimensional, and quadratic knapsack problems," Comput. Oper. Res., vol. 143, Art. no. 105693, 2022.

[21] K. Bringmann, "Knapsack with small items in near-quadratic time," in Proc. 56th Annu. ACM Symp. Theory Comput. (STOC), 2024.

[22] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," Nature Methods, vol. 17, pp. 261-272, 2020.

[23] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357-362, 2020.

[24] C. She et al., "A tutorial on ultrareliable and low-latency communications in 6G: Integrating domain knowledge into deep learning," Proc. IEEE, vol. 109, no. 3, pp. 204-246, Mar. 2021.

[25] Python Software Foundation, "concurrent.futures - Launching parallel tasks," Python 3 documentation. [Online]. Available: https://docs.python.org/3/library/concurrent.futures.html (accessed Oct. 2026).